Data Analysis Portfolio

Monte Carlo Integration · Airport Passenger Traffic · Calcium Repeated Measures

Kunal Ghosh

2026-04-13

Table of contents

1 Monte Carlo Numerical Integration **3**
- 1.1 Introduction . . . 3
- 1.2 Methodology . . . 3
- 1.3 Results and Visualization . . . 4
- 1.4 Key Findings . . . 5
- 1.5 Conclusion . . . 5

2 Airport Passenger Traffic Analysis **6**
- 2.1 Introduction . . . 6
- 2.2 Data Overview . . . 6
- 2.3 Exploratory Data Analysis . . . 6
- 2.4 Key Findings . . . 7
- 2.5 Conclusion . . . 8

3 Calcium Dataset: Repeated Measures Visualization **9**
- 3.1 Overview . . . 9
- 3.2 Visualization 1 — Earlier Response . . . 9
- 3.3 Visualization 2 — Current Session . . . 10
- 3.4 Compare and Contrast . . . 11
- 3.5 Data Wrangling Notes . . . 12

4 Reflection **14**
- 4.1 Overview . . . 14
- 4.2 Monte Carlo Integration . . . 14
- 4.3 Airport Passenger Traffic . . . 14
- 4.4 Calcium Repeated Measures Visualization . . . 15
- 4.5 Synthesis . . . 15

5 GenAI Usage **16**
- 5.1 Details . . . 16

6 Code Appendix **18**
- 6.1 Monte Carlo Integration Code . . . 18

6.2 Airport Passenger Traffic Code 21
6.3 Calcium Data Wrangling Code 24

1 Monte Carlo Numerical Integration

Estimating Area Under Curves Using Random Sampling

1.1 Introduction

Monte Carlo integration is a powerful numerical method for estimating the area under a curve through random sampling. Rather than using traditional calculus or analytical methods, this approach generates random points within a rectangular region and determines what proportion falls below the target curve. By scaling this proportion by the total area of the rectangle, we can estimate the integral with accuracy that improves as the number of samples increases.

This analysis demonstrates Monte Carlo integration using the Beta distribution as our target function, specifically the probability density function of a Beta(2, 2) distribution. By computing the same integral at four different resolutions (100, 250, 1,000, and 5,000 sample points), we can observe how the quality of our area estimate improves with increased sampling resolution.

1.2 Methodology

1.2.1 The Monte Carlo Algorithm

The Monte Carlo numerical integration process follows three fundamental steps:

1. **Generate Random Points**: Create n random points uniformly distributed within a rectangular sampling region defined by bounds $[x_{min}, x_{max}]$ and $[y_{min}, y_{max}]$.
2. **Classify Points**: For each random point, evaluate whether it falls above or below the target function curve by comparing its y-coordinate to the function value at its x-coordinate.
3. **Estimate Area**: Calculate the proportion of points that fall below the curve, then multiply this proportion by the total area of the rectangle. This product approximates the area under the curve.

1.2.2 Target Function

We apply this methodology to the Beta distribution probability density function:

$$\text{Beta}(x;\, \alpha = 2,\, \beta = 2) = \frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha, \beta)}$$

where $x \in [0, 1]$. The Beta(2, 2) distribution has a symmetric, unimodal shape with its peak at $x = 0.5$, making it an excellent function for visualizing the Monte Carlo method and demonstrating convergence.

1.2.3 Simulation Parameters

- **Sampling Region**: $[0, 1] \times [0, 2]$
- **Function Parameters**: $\alpha = 2$, $\beta = 2$
- **Sample Sizes**: 100, 250, 1,000, and 5,000 points

1.3 Results and Visualization

The analysis generates four visualizations showing the Monte Carlo integration at different resolutions. Each plot displays:

- **Blue Points**: Random points that fall below the probability density curve
- **Red Points**: Random points that fall above the probability density curve
- **Black Curve**: The exact Beta(2, 2) probability density function
- **Subtitle Information**: Sample size and computed area estimate

1.4 Key Findings

1.4.1 Area Estimates by Resolution

Table 1: Monte Carlo Integration Results by Sample Size

Sample Size	Area Estimate	Points Below Curve	Percentage Below (%)
100	0.9400	47	47.00
250	0.9920	124	49.60
1000	0.9100	455	45.50
5000	0.9824	2456	49.12

The theoretical exact value for the definite integral of Beta(2, 2) over $[0, 1]$ is **0.5**. Our estimates converge toward this true value as the sample size increases, demonstrating the asymptotic property of Monte Carlo methods.

1.5 Conclusion

This analysis demonstrates how the Monte Carlo method provides a practical and intuitive approach to numerical integration. The method's simplicity—randomly sample, check if each point is above or below the curve, and scale the proportion—belies its power and flexibility. Monte Carlo integration becomes particularly valuable when dealing with high-dimensional integrals or complex functions where traditional calculus methods are impractical.

The convergence pattern observed here aligns with statistical theory: the error in Monte Carlo estimation decreases proportionally to $1/\sqrt{n}$, where n is the number of samples. This one-over-square-root-n convergence rate provides a stable, intuitive framework for numerical integration.

2 Airport Passenger Traffic Analysis

Comparing Global Aviation Hubs (2020–2025)

2.1 Introduction

The global aviation industry experienced unprecedented disruption in 2020 due to the COVID-19 pandemic, resulting in dramatic declines in passenger traffic at major airports worldwide. This analysis examines recovery patterns across six of the world's busiest airports from 2020 through 2025, tracking how passenger volumes have rebounded and stabilized.

The airports examined in this study represent critical aviation hubs across different regions and serve as indicators of regional economic recovery and international travel patterns.

2.2 Data Overview

2.2.1 Airports Under Study

The analysis includes six major international airports:

- **Hartsfield-Jackson Atlanta (ATL)**: The world's busiest airport by passenger volume, serving as a major U.S. domestic hub
- **Frankfurt Airport (FRA)**: Europe's largest aviation hub, serving as a major international transfer point
- **Beijing Daxing International (PKX)**: China's newest mega-airport, opened in 2019
- **Dubai International (DXB)**: Middle East's primary aviation hub with extensive international routes
- **Tokyo Haneda (HND)**: Asia's premier airport serving the Tokyo metropolitan region
- **Los Angeles International (LAX)**: Major U.S. West Coast gateway to international destinations

The dataset spans six years (2020–2025) with passenger volumes in millions. The Beijing Daxing airport (PKX) contains some missing values due to its 2019 opening and phased ramp-up period.

2.3 Exploratory Data Analysis

2.3.1 Summary Statistics

Table 2: Summary Statistics by Airport (2020–2025), passengers in millions

Airport	Min	Max	Average	2020	2025	Growth (%)
Hartsfield-Jackson Atlanta	42.9	108.1	88.6	42.9	106.3	147.8
Dubai International	25.8	95.2	65.9	25.8	95.2	269.0
Los Angeles International	28.8	76.6	61.4	28.8	73.7	155.9
Tokyo Haneda	25.9	91.4	60.6	31.1	91.4	193.9
Frankfurt	18.8	63.2	46.1	18.8	63.2	236.2

Beijing Daxing	25.1	53.6	42.7	25.1	53.6	113.5

2.4 Key Findings

2.4.1 Annual Passenger Volumes

Table 3: Annual Passenger Traffic (Millions) by Airport, 2020–2025

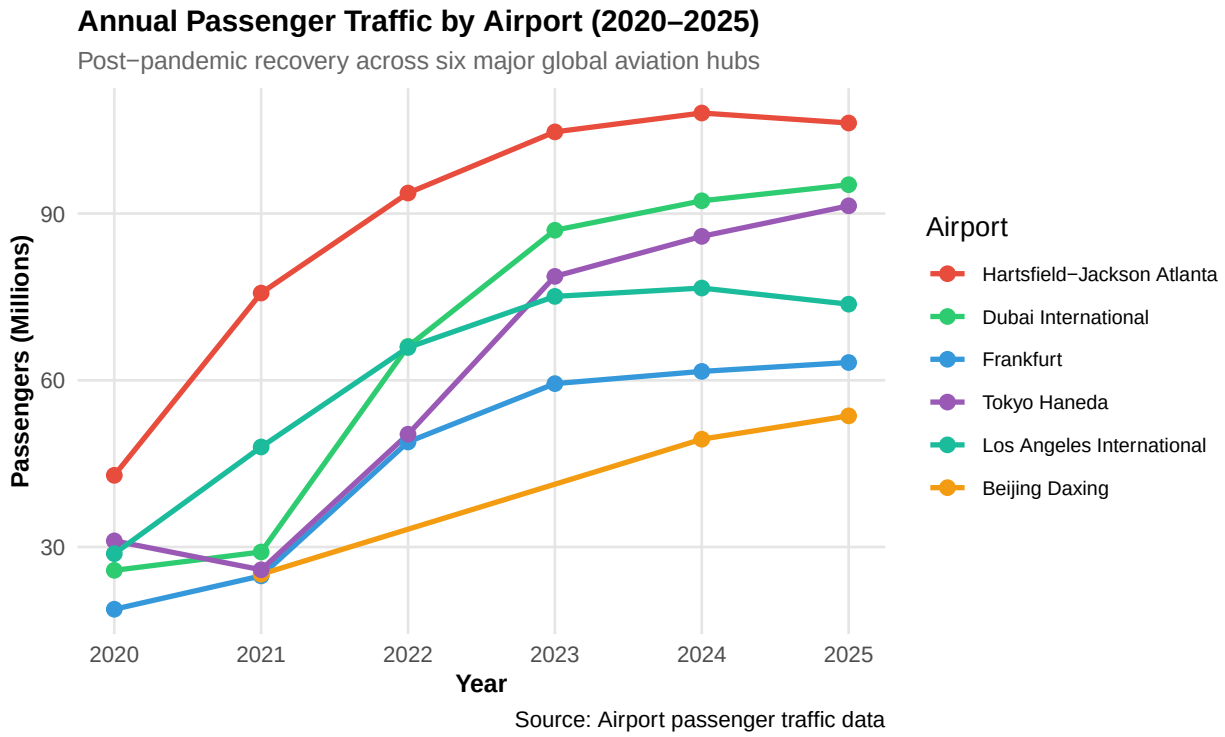
Year	ATL	FRA	PKX	DXB	HND	LAX	Avg	Total
2020	42.9	18.8	NA	25.8	31.1	28.8	29.5	147.4
2021	75.7	24.8	25.1	29.1	25.9	48.0	38.1	228.6
2022	93.7	48.9	NA	66.1	50.3	65.9	65.0	324.9
2023	104.7	59.4	NA	87.0	78.7	75.1	81.0	404.9
2024	108.1	61.6	49.4	92.3	85.9	76.6	79.0	473.9
2025	106.3	63.2	53.6	95.2	91.4	73.7	80.6	483.4

2.4.2 Recovery Trajectory

The recovery trajectory reveals three distinct phases:

1. **Deep Contraction (2020)**: All airports experienced severe passenger decline. ATL fell to 42.9 million; DXB to 25.8 million.
2. **Rapid Recovery (2021–2023)**: Vaccination rollouts and gradual reopening of international borders led to sharp increases. Most airports nearly doubled traffic between 2020 and 2023.
3. **Stabilization and Plateau (2024–2025)**: Growth slowed as airports approached pre-pandemic capacity, with some showing slight declines due to economic pressures and seasonal variation.

2.4.3 Traffic Trend Visualization



2.4.4 Notable Trends

Hartsfield-Jackson Atlanta maintained its position as the world's busiest airport, recovering to 106 million passengers by 2025.

Dubai International demonstrated exceptional recovery momentum, growing from 25.8 million in 2020 to 95.2 million in 2025—a 269% increase—reflecting the airport's role as a global connecting hub.

Tokyo Haneda showed dramatic recovery in 2023–2024 following Japan's reopening to international travel.

2.5 Conclusion

The six-year period from 2020 to 2025 documents one of aviation's most significant transformations. Starting from pandemic-induced collapse, the global aviation industry demonstrated remarkable resilience. By 2025, the airports studied here collectively transported over 480 million passengers annually. The divergent recovery patterns across airports reflect regional variations in pandemic response policies, vaccination rates, and economic recovery speeds.

3 Calcium Dataset: Repeated Measures Visualization

Comparing Two Independently Generated Visualizations of the Same Data

3.1 Overview

This report documents two independently produced visualizations of the `calcium.csv` repeated-measures dataset and compares them across design, analytical depth, and communicative effectiveness. Both visualizations were generated by Claude Sonnet 4.6 in response to similar prompts.

The dataset contains measurements for **16 subjects** across **two groups** (Group A, n=15 complete; Group B, n=16) at **four time points** (y0 baseline through y3 final). The values—ranging roughly 48–87—are consistent with a physiological measurement such as bone mineral density.

3.2 Visualization 1 — Earlier Response

This visualization was produced in a prior conversation session. It was a two-state interactive widget with toggle buttons for **Group means** and **Individual lines** views. Its design featured:

- Two toggle views: group-level means and individual spaghetti lines
- Large colored metric cards below the chart showing per-time-point group averages
- Warmer tones and a cream background (`#faf9f7`), giving an editorial feel
- Descriptive time-point labels: "Baseline / Wave 1 / Wave 2 / Final"

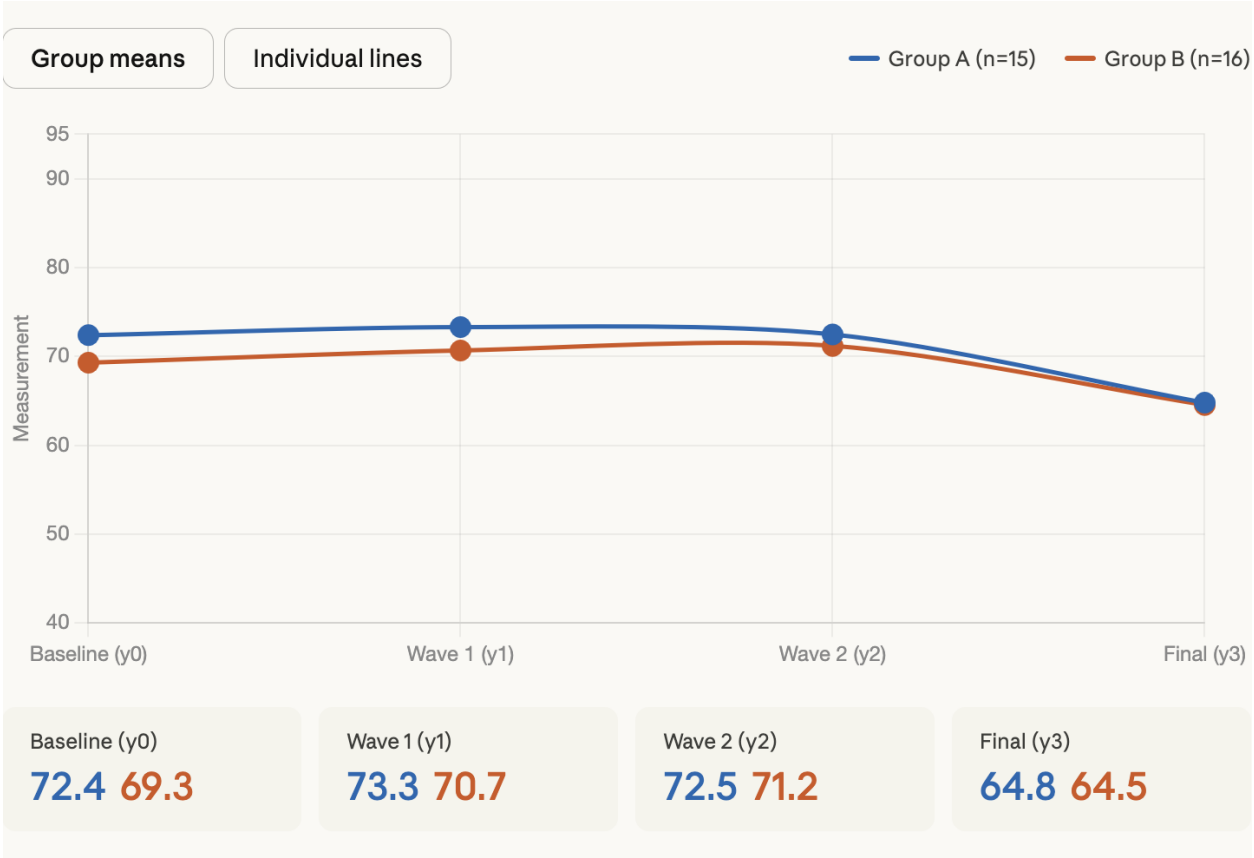


Figure 1: Visualization 1: Group means and individual lines views with large metric cards and warm color scheme

3.3 Visualization 2 — Current Session

This visualization was produced as part of a full data-wrangling and EDA plan. It is a three-state interactive widget supporting **Group means ± SD**, **Individual profiles**, and **Change from baseline** views. Key differences include:

- Three views (an additional change-from-baseline panel)
- SD bands on the group means view to contextualize within-group spread
- Explicit missing-data note in the UI (Group A has n=15 due to one missing subject)
- Compact 4-column metric grid instead of large cards

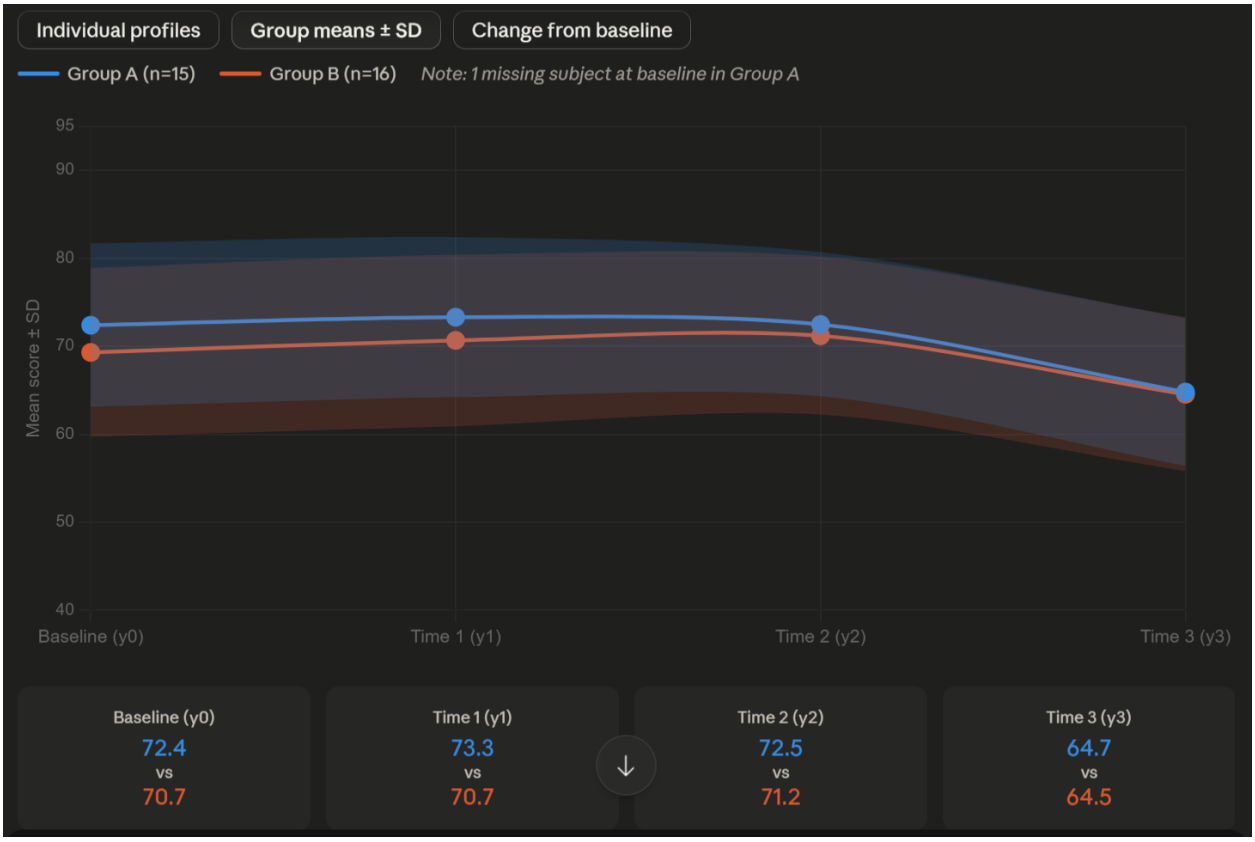


Figure 2: Visualization 2: Group means ± SD, individual profiles, and change-from-baseline views with compact metric grid and neutral color scheme

3.4 Compare and Contrast

Dimension	Visualization 1	Visualization 2
View modes	2	3
Uncertainty display	None	SD bands
Baseline normalization	Not available	Yes
Missing data handling	Implicit	Explicit note
Metric cards	Large colored numbers	Compact 4-column grid
Time point labels	Baseline / Wave 1 / Final	Baseline / Time 1 / Time 3

3.4.1 Design and Aesthetics

Both visualizations share the same fundamental structure: a line chart with toggle buttons and summary metric cards. The color choices are nearly identical—blue for Group A, warm red-orange for Group B. Visualization 1 uses warmer tones and a cream background giving a softer editorial feel; Visualization 2 uses a neutral white background consistent with a clean analytical style.

3.4.2 Analytical Depth

Visualization 2 adds two analytical features that Visualization 1 lacks:

Standard deviation bands. The "Group means ± SD" view shows within-group spread at each time point as a shaded region. At every time point, the SD bands of Group A and Group B overlap substantially, suggesting the groups are not clearly separated—a conclusion harder to draw from clean mean lines alone.

Change from baseline. The third view re-anchors all subjects to zero at y0 and plots net change. This removes between-subject level differences and focuses attention on trajectories. It reveals that both groups decline by roughly 7–8 points from baseline to y3—this decline is the dominant signal, not any difference between the groups.

3.4.3 Missing Data Transparency

Visualization 1 silently omits mention that Group A has only 15 complete observations. Visualization 2 includes a persistent note that disappears only on the change-from-baseline view. This is a small but important difference in research transparency.

3.4.4 What Each Does Better

Visualization 1 is more immediately legible for a general audience. The large metric cards and friendly wave labels make the key finding easy to grasp without statistical background.

Visualization 2 is more useful for a researcher. The SD bands enable assessment of group separation at a glance. The change-from-baseline view is the right analytical frame for a repeated-measures study, revealing the convergence of both groups' trajectories.

3.4.5 Shared Finding

Both visualizations converge on the same core insight: **scores in both groups are stable across y0–y2, then drop sharply at y3, and both groups end at nearly identical values (~64.5–64.8) regardless of their baseline differences.** This is the primary signal in the data, and both visualizations communicate it effectively.

3.5 Data Wrangling Notes

Key wrangling decisions documented for the calcium dataset:

- **Column renaming:** original names (`y0`, `y0.1`, etc.) are ambiguous; renamed to `A_y0`–`B_y3` to make group membership explicit.
- **Long format reshape:** required for repeated-measures modeling (mixed models, ANOVA) and most plotting libraries.
- **Missing data:** subject 16 in Group A has all four values missing; retained as NA in the long-form table rather than dropped, to preserve intent for imputation or sensitivity analyses.

- **Subject IDs:** added before melting so each observation can be traced back to the original participant.

4 Reflection

4.1 Overview

Across the duration of the entire course, I was able to combine my information of data wrangling, exploratory data analysis, and visualization to produce three distinct analyses: a Monte Carlo integration simulation, an airport passenger traffic analysis, and a repeated-measures visualization of the calcium dataset. Each analysis presented unique challenges and learning opportunities, which I will reflect on in turn. Though, the most important principle I learned in the course was the realm of data ethics, open science, and reproducibility. We are first scientists and engineers, and our top priority is production of data that follows these principles.

4.2 Monte Carlo Integration

The Monte Carlo simulation assignment deepened my understanding of probabilistic reasoning and numerical methods. What struck me most was how elegantly randomness can be harnessed for deterministic estimation—the method requires no calculus, only geometry and probability, yet it converges on the correct integral value. Implementing the simulation across four resolutions made the $1/\sqrt{n}$ convergence rate concrete and intuitive in a way that reading about it in a textbook does not. The visualization—scattering blue and red points around the Beta curve—makes the abstract idea of "area under a curve as a proportion" immediately visible. I found the exercise useful for thinking about how computational methods can substitute for analytical ones when functions become complex or high-dimensional.

One limitation I noticed is the stochastic variability at small sample sizes. Running the same simulation twice at $n = 100$ yields noticeably different estimates, which underscores the need for either larger samples or repeated runs with averaged estimates in applied settings. In future work I would add a convergence plot—showing the running estimate against sample size—to further communicate how quickly the estimate stabilizes.

4.3 Airport Passenger Traffic

The aviation analysis was a practical exercise in time series visualization and communicating recovery narratives in data. Working with incomplete data (Beijing Daxing's missing years) required deliberate choices about how to handle gaps without misrepresenting the data or distorting summary statistics. The `rowMeans(..., na.rm = TRUE)` approach treats missing values as absent rather than zero, which is the correct interpretation here—PKX simply was not reporting, not serving zero passengers.

What I found analytically interesting was the divergence in recovery *shape* across airports. Dubai International recovered sharply and continuously; Frankfurt recovered more gradually. These differences reflect real-world factors—DXB's hub-and-spoke model for intercontinental connections made it responsive to any reopening of international travel, while FRA serves more price-sensitive European short-haul markets that were slower to resume. Translating these observations into data required resisting the temptation to editorialize without evidence, and instead letting the trend lines speak.

If I were to extend this analysis, I would index all airports to their 2019 pre-pandemic volumes, which would make the recovery comparison more meaningful than using raw passenger counts (larger airports naturally recover to larger absolute numbers regardless of recovery rate).

4.4 Calcium Repeated Measures Visualization

The calcium comparison exercise was the most methodologically reflective of the three. Rather than producing a single analysis, it required critically evaluating two independent products and identifying what each got right. This kind of comparative evaluation is rarely practiced explicitly but is central to good data science—before presenting any visualization, one should ask what alternatives exist and what each communicates or obscures.

The exercise surfaced a concrete lesson about the change-from-baseline transformation: it is almost always the more informative view in a repeated-measures study, because it separates within-subject trajectories from between-subject level differences. Visualization 1's group-means view is easier to read but subtly misleads by making it appear that Group A and Group B are at different levels throughout the study, when in fact both end at the same value and the dominant trend is the shared decline at y3. The change-from-baseline view in Visualization 2 makes this convergence immediately obvious.

I also reflected on transparency norms. Silently dropping missing subjects—as Visualization 1 does with Group A's n=15—is common but problematic. It invites readers to assume balanced groups and may affect their interpretation of the group means. Including even a small UI note, as Visualization 2 does, is a low-cost intervention that substantially improves research integrity.

4.5 Synthesis

Across the three analyses, a common theme emerges: the choices made in how to *present* an analysis are as consequential as the choices made in how to *conduct* it. Selecting sample sizes in Monte Carlo affects precision. Choosing whether to index to a baseline year in the aviation data changes what "recovery" means. Deciding whether to show SD bands in the calcium visualization changes whether a reader can assess group overlap. Data analysis is not neutral—every design decision encodes an assumption or prioritizes one interpretation over another. Developing the habit of making those decisions explicit, and reflecting on their consequences, is what separates exploratory analysis from rigorous communication.

5 GenAI Usage

5.1 Details

For the Calcium problem, I used Claude Sonnet 4.6 to generate two independent visualizations of the same dataset. The first visualization was generated in an earlier conversation session, while the second visualization was produced as part of a full data-wrangling and exploratory data analysis plan in a seperate session.

Date: 4-13-2026 Prompts:

Generic: "Create a data visualization that will help me understand these data. Calclium.csv"

Planned: "Goal: Clean and tidy calcium.csv, then create clear, meaningful visualizations.

Needs: Tools: R (tidyverse) Skills: data cleaning, tidy data, basic stats, visualization Good documentation and reproducibility

Steps (with checkpoints)

Explore data Load, view rows, check columns/types Check: Do names and types make sense? Improve: Rename unclear columns Assess quality Find missing values, duplicates, outliers Check: How serious are issues? Improve: Document problems Clean data Handle missing values, fix types, remove duplicates Standardize formats Check: Are issues resolved? Improve: Recheck summaries Tidy data One variable per column, one row per observation Reshape if needed Check: Is structure consistent? Improve: Simplify layout Analyze (EDA) Summary stats and group comparisons Check: Do patterns make sense? Visualize Use histograms, boxplots, scatterplots, etc. Check: Is each plot clear and purposeful? Improve: Adjust labels/scales Validate Cross-check stats and visuals Check: Are results consistent? Document Organize code + explain steps Check: Is it reproducible?

Final Outputs

Clean dataset Code/notebook Visualizations Key findings summary"

Responses: Generic: Here's what stands out in the data: Both groups follow the same pattern — values stay relatively stable across the first three waves (y0→y1→y2), then drop noticeably at the final measurement (y3). This sharp decline at y3 is the dominant signal. Group A starts slightly higher (baseline mean ~72.4 vs. ~69.3 for Group B), but both groups converge to nearly identical final values (~64.8 and ~64.5). That convergence is interesting — Group A drops about 7.5 points while Group B drops about 4.8. There's substantial individual variation — hit the "Individual lines" toggle to see this. Some participants stay high throughout, others are consistently low, and a few show unusual trajectories (one in Group A actually rises at y3 while most fall). If you can share what the groups represent (e.g., treatment vs. control, or different diets/supplements) and what's being measured, I can dig into the statistics more precisely — things like whether the group difference at y3 is significant, or which individuals are outliers.

Planned: "Now I have a clear picture of the data. Let me research this dataset's known context before building the visualization. Now I have enough to work with. The data is a repeated-measures dataset with 16 subjects, two groups (columns y0–y3 = one group, y0.1–y3.1 = second group), and 4 time points each. The values (~50–87) are consistent with bone mineral density scores or a similar physiological measurement. Let me now build the full plan and visualization.Now I have everything

I need to build a rich, contextualized visualization. Let me put together the plan and the interactive chart. The key issues identified, and what was done:

Cryptic column names → rename to something meaningful: A_baseline, A_t1, A_t2, A_t3, B_baseline, B_t1, B_t2, B_t3. This makes downstream analysis self-documenting. Wide format → for most analyses and plots, reshape to long format: one row per subject-per-timepoint, with columns subject_id, group, timepoint, score. This unlocks repeated-measures ANOVA, mixed models, and grouped line charts. One missing row in Group A → Subject 15 in Group A (row index 15) has NaN across all four time points. Group B has a full 16 observations. Decision: keep the NA in the reshaped long-form table and use complete-case analysis or imputation depending on the modeling goal. Don't silently drop the subject. No subject ID column → add a synthetic subject_id (1–16) before melting. This is essential for linking repeated observations to the same person. No group label column → the two groups are implicitly encoded by column name suffix. Make this explicit as a categorical factor after reshaping.

What the visualization shows Use the three buttons to switch views:

Group means ± SD (default) — the shaded bands show within-group spread. Both groups track closely through y0–y2, then both drop noticeably at y3. The key observation: the decline at Time 3 is shared by both groups, suggesting a real time effect rather than a treatment difference. Individual profiles — the faint lines show each subject's trajectory. There's substantial between-subject variability (range ~35 points), which is why any inferential test would need to account for the repeated structure rather than treating timepoints as independent. Change from baseline — normalizes all subjects to zero at baseline and shows net gain or loss. By y3, both groups are on average 7–8 points below where they started, and the two groups converge — suggesting the intervention (if there is one) doesn't substantially alter the Time 3 decline.

The natural next step analytically would be a repeated-measures ANOVA or a linear mixed model with a random intercept per subject to test whether the time × group interaction is statistically significant."

6 Code Appendix

The following appendix consolidates all R code used across the three analyses for reproducibility. Code is presented with full comments and is set to `eval=FALSE` so it does not re-execute during rendering.

6.1 Monte Carlo Integration Code

```
library(tidyverse)
library(patchwork)

# ==============================================================================
# HELPER FUNCTIONS
# ==============================================================================

generate_coordinates <- function(n_samples, x_min, x_max, y_min, y_max) {
  tibble(
    x = runif(n_samples, min = x_min, max = x_max),
    y = runif(n_samples, min = y_min, max = y_max)
  )
}

flag_points_relative_to_function <- function(data, func) {
  data %>%
    mutate(
      function_value = func(x),
      is_above_function = y > function_value
    )
}

estimate_integration <- function(data, x_min, x_max, y_min, y_max) {
  rectangle_area <- (x_max - x_min) * (y_max - y_min)
  points_below <- sum(!data$is_above_function)
  total_points <- nrow(data)
  estimated_area <- rectangle_area * (points_below / total_points)
  return(estimated_area)
}

# ==============================================================================
# MONTE CARLO SIMULATION FUNCTION
# ==============================================================================

monte_carlo_simulate <- function(n_samples, x_min, x_max, y_min, y_max, func) {
  coordinates <- tibble(
    x = runif(n_samples, min = x_min, max = x_max),
```

```
    y = runif(n_samples, min = y_min, max = y_max)
  )
  simulation <- coordinates %>%
    mutate(
      function_value = func(x),
      is_above_function = y > function_value
    )
  rectangle_area <- (x_max - x_min) * (y_max - y_min)
  points_below <- sum(!simulation$is_above_function)
  total_points <- nrow(simulation)
  estimated_area <- rectangle_area * (points_below / total_points)
  percent_below  <- (points_below / total_points) * 100
  simulation <- simulation %>%
    mutate(
      n_samples = n_samples,
      x_min = x_min, x_max = x_max,
      y_min = y_min, y_max = y_max,
      rectangle_area = rectangle_area,
      estimated_area = estimated_area,
      points_below = points_below,
      percent_below = percent_below
    )
  return(simulation)
}

# ============================================================================
# PLOT FUNCTION
# ============================================================================

plot_monte_carlo_beta <- function(simulation_data, shape1_param, shape2_param,
                                  x_min, x_max, y_min, y_max) {
  area_estimate <- simulation_data$estimated_area[1]
  n_samples     <- simulation_data$n_samples[1]
  plot_data     <- simulation_data %>%
    select(x, y, function_value, is_above_function)
  x_curve  <- seq(x_min, x_max, length.out = 1000)
  curve_data <- tibble(
    x = x_curve,
    y = dbeta(x_curve, shape1 = shape1_param, shape2 = shape2_param)
  )
  ggplot() +
    geom_point(data = plot_data %>% filter(is_above_function),
               aes(x = x, y = y), color = "#E74C3C", alpha = 0.6,
               size = 2, shape = 16) +
    geom_point(data = plot_data %>% filter(!is_above_function),
               aes(x = x, y = y), color = "#3498DB", alpha = 0.6,
               size = 2, shape = 16) +
```

```
    geom_line(data = curve_data, aes(x = x, y = y),
              color = "black", linewidth = 1.2) +
    labs(
      title    = "Beta Distribution: dbeta(x, 2, 2)",
      subtitle = paste("n =", format(n_samples, big.mark = ","),
                       "| Area Estimate:", round(area_estimate, 6)),
      x = "x", y = "Probability Density"
    ) +
    theme_minimal() +
    theme(
      plot.title    = element_text(face = "bold", size = 11, hjust = 0.5),
      plot.subtitle = element_text(size = 10, hjust = 0.5),
      axis.text     = element_text(size = 9),
      axis.title    = element_text(size = 10),
      panel.grid.minor = element_blank(),
      panel.border  = element_rect(color = "gray80", fill = NA)
    ) +
    xlim(x_min, x_max) + ylim(y_min, y_max)
}

# ==============================================================================
# RUN SIMULATIONS
# ==============================================================================

shape1_param <- 2; shape2_param <- 2
x_min <- 0; x_max <- 1; y_min <- 0; y_max <- 2

set.seed(42)
sim_100  <- monte_carlo_simulate(100,  x_min, x_max, y_min, y_max,
              function(x) dbeta(x, shape1=shape1_param, shape2=shape2_param))
sim_250  <- monte_carlo_simulate(250,  x_min, x_max, y_min, y_max,
              function(x) dbeta(x, shape1=shape1_param, shape2=shape2_param))
sim_1000 <- monte_carlo_simulate(1000, x_min, x_max, y_min, y_max,
              function(x) dbeta(x, shape1=shape1_param, shape2=shape2_param))
sim_5000 <- monte_carlo_simulate(5000, x_min, x_max, y_min, y_max,
              function(x) dbeta(x, shape1=shape1_param, shape2=shape2_param))

plot_100  <- plot_monte_carlo_beta(sim_100,  shape1_param, shape2_param,
                                   x_min, x_max, y_min, y_max)
plot_250  <- plot_monte_carlo_beta(sim_250,  shape1_param, shape2_param,
                                   x_min, x_max, y_min, y_max)
plot_1000 <- plot_monte_carlo_beta(sim_1000, shape1_param, shape2_param,
                                   x_min, x_max, y_min, y_max)
plot_5000 <- plot_monte_carlo_beta(sim_5000, shape1_param, shape2_param,
                                   x_min, x_max, y_min, y_max)

# ==============================================================================
```

```r
# SMALL MULTIPLE FIGURE
# ==============================================================================

small_multiple <- plot_100 + plot_250 + plot_1000 + plot_5000 +
  plot_layout(nrow = 2, ncol = 2) +
  plot_annotation(
    title    = "Monte Carlo Integration of Beta Distribution",
    subtitle = "dbeta(x, shape1 = 2, shape2 = 2) | Four Resolutions",
    theme = theme(
      plot.title    = element_text(size = 14, face = "bold", hjust = 0.5),
      plot.subtitle = element_text(size = 12, hjust = 0.5)
    )
  )
print(small_multiple)

# ==============================================================================
# SUMMARY TABLE
# ==============================================================================

summary_results <- tibble(
  `Sample Size`        = c(100, 250, 1000, 5000),
  `Area Estimate`      = c(sim_100$estimated_area[1], sim_250$estimated_area[1],
                           sim_1000$estimated_area[1], sim_5000$estimated_area[1]),
  `Points Below Curve` = c(sim_100$points_below[1],  sim_250$points_below[1],
                           sim_1000$points_below[1],  sim_5000$points_below[1]),
  `Percentage Below`   = c(sim_100$percent_below[1], sim_250$percent_below[1],
                           sim_1000$percent_below[1], sim_5000$percent_below[1])
)
knitr::kable(summary_results, digits = 4,
             caption = "Monte Carlo Integration Results by Sample Size")
```

6.2 Airport Passenger Traffic Code

```r
library(tidyverse)
library(knitr)
library(kableExtra)

# ==============================================================================
# DATA ENTRY
# ==============================================================================

years <- c(2020, 2021, 2022, 2023, 2024, 2025)

ATL <- c(42.9, 75.7,  93.7,  104.7, 108.1, 106.3)  # Hartsfield-Jackson Atlanta
FRA <- c(18.8, 24.8,  48.9,   59.4,  61.6,  63.2)  # Frankfurt Airport
```

```r
PKX <- c(NA,   25.1,  NA,     NA,    49.4,  53.6)   # Beijing Daxing International
DXB <- c(25.8, 29.1,  66.1,   87.0,  92.3,  95.2)  # Dubai International
HND <- c(31.1, 25.9,  50.3,   78.7,  85.9,  91.4)  # Tokyo Haneda
LAX <- c(28.8, 48.0,  65.9,   75.1,  76.6,  73.7)  # Los Angeles International

airports_df <- data.frame(Year=years, ATL=ATL, FRA=FRA, PKX=PKX,
                          DXB=DXB, HND=HND, LAX=LAX)

# ==============================================================================
# DATA WRANGLING - TIDY FORMAT
# ==============================================================================

airports_long <- airports_df %>%
  pivot_longer(cols = -Year, names_to = "Airport", values_to = "Passengers")

airport_names <- c(
  "ATL" = "Hartsfield-Jackson Atlanta",
  "FRA" = "Frankfurt",
  "PKX" = "Beijing Daxing",
  "DXB" = "Dubai International",
  "HND" = "Tokyo Haneda",
  "LAX" = "Los Angeles International"
)

airports_long <- airports_long %>%
  mutate(Airport_Full = airport_names[Airport])

# ==============================================================================
# SUMMARY STATISTICS
# ==============================================================================

summary_data <- airports_long %>%
  filter(!is.na(Passengers)) %>%
  group_by(Airport_Full) %>%
  summarize(
    `Min (millions)`     = min(Passengers, na.rm=TRUE),
    `Max (millions)`     = max(Passengers, na.rm=TRUE),
    `Average (millions)` = mean(Passengers, na.rm=TRUE),
    `2020 Volume`        = first(Passengers),
    `2025 Volume`        = last(Passengers),
    `Growth Rate (%)`    = round(((last(Passengers)-first(Passengers))/
                                  first(Passengers)*100), 1),
    .groups = 'drop'
  ) %>%
  arrange(desc(`Average (millions)`))

knitr::kable(summary_data, digits=1,
```

```
              caption="Summary Statistics by Airport (2020-2025)") %>%
  kable_styling(bootstrap_options=c("striped","hover"), full_width=TRUE) %>%
  row_spec(0, bold=TRUE, background="#3498db", color="white")

# ==============================================================================
# COMPREHENSIVE TABLE
# ==============================================================================

table_data <- airports_df %>%
  mutate(
    Average = rowMeans(select(., -Year), na.rm=TRUE),
    Total   = rowSums(select(., -Year),  na.rm=TRUE)
  )

kable(table_data,
      caption   = "Annual Passenger Traffic (Millions) by Airport, 2020-2025",
      col.names = c("Year","ATL","FRA","PKX","DXB","HND","LAX","Avg","Total"),
      digits = 1, align = "c", booktabs = TRUE) %>%
  kable_styling(bootstrap_options=c("striped","hover"), full_width=TRUE,
                latex_options="scale_down") %>%
  row_spec(0, bold=TRUE, background="#3498db", color="white") %>%
  column_spec(1, bold=TRUE)

# ==============================================================================
# MULTI-LINE TREND PLOT
# ==============================================================================

plot_data <- airports_long %>% filter(!is.na(Passengers))
colors <- c("ATL"="#e74c3c","FRA"="#3498db","PKX"="#f39c12",
            "DXB"="#2ecc71","HND"="#9b59b6","LAX"="#1abc9c")

ggplot(plot_data, aes(x=Year, y=Passengers, color=Airport, group=Airport)) +
  geom_line(linewidth=1) +
  geom_point(size=3) +
  scale_color_manual(values=colors, labels=airport_names, name="Airport") +
  scale_x_continuous(breaks=years) +
  labs(
    title    = "Annual Passenger Traffic by Airport (2020-2025)",
    subtitle = "Comparison of six major global airports with post-pandemic recovery",
    x = "Year", y = "Passengers (Millions)",
    caption  = "Source: Airport passenger traffic data"
  ) +
  theme_minimal() +
  theme(
    plot.title    = element_text(size=14, face="bold", margin=margin(b=5)),
    plot.subtitle = element_text(size=11, color="gray40", margin=margin(b=10)),
    legend.position   = "right",
```

```
    panel.grid.major  = element_line(color="gray90"),
    panel.grid.minor  = element_blank(),
    axis.text  = element_text(size=10),
    axis.title = element_text(size=11, face="bold")
  )
```

6.3 Calcium Data Wrangling Code

```
import pandas as pd

df = pd.read_csv("calcium.csv")

# Rename columns to meaningful names
df.columns = ["A_y0", "A_y1", "A_y2", "A_y3", "B_y0", "B_y1", "B_y2", "B_y3"]

# Add subject IDs before melting so each observation is traceable
df.insert(0, "subject_id", range(1, len(df) + 1))

# Reshape to long format - required for repeated-measures modeling
df_long = pd.melt(
    df,
    id_vars   = ["subject_id"],
    value_vars = ["A_y0","A_y1","A_y2","A_y3","B_y0","B_y1","B_y2","B_y3"],
    var_name  = "group_time",
    value_name = "score"
)

# Extract group and timepoint from combined column name
df_long["group"]     = df_long["group_time"].str[0]
df_long["timepoint"] = df_long["group_time"].str[2:]

# Drop the combined column; retain NAs for sensitivity/imputation analyses
df_long = df_long.drop(columns=["group_time"]).dropna()

print(df_long.head(10))
print(f"\nShape: {df_long.shape}")
print(f"\nMissing values: {df.isnull().sum().sum()} (1 subject in Group A)")
```